

BANK ACCOUNT MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of
the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

M.DEDIPYA– 25B11AI767

P.DURGA DEVI - 25B11AI971

K.CHINMAI – 25B11AI548

P.SASI – 25B11AI874

Under the esteemed guidance of

Mr. T.Srinivasulu M.Tech, (Ph.D.)

Assistant Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 – 2027

ADITYA UNIVERSITY

(Formerly Aditya Engineering College(A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “BANK ACCOUNT MANAGEMENT SYSTEM” is being submitted by M.Dedipya (Roll No.25B11AI767), P.Durgadevi (Roll No.25B11AI971), K.Chinmai (Roll No. 25B11AI548) and P.Sasi (Roll No. 25B11AI874) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

Project Guide

Mr. T. Srinivasulu, M. Tech, (Ph.D.)
Assistant Professor

Head of the Department

Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

External Examiner

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “BANK ACCOUNT MANAGEMENT SYSTEM” is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Mr. T. Srinivasulu, M.Tech, (Ph.D.), Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

M. Dedipya – _ (25B11AI767)

P. Durga Devi – (25B11AI971)

K. Chinmai - (25B11AI548)

P. Sasi – (25B11AI874_)

Date: 30 September 2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, M.Tech, (Ph.D.), Department of _ Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, M.Tech, Ph.D, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

M. Dedipya — 25B11AI767

P. Durga Devi — 25B11AI971

K. Chinmai — 25B11AI548

P. Sasi — 25B11AI874

ABSTRACT

The Bank Account Management System is a database-based application developed to organize, store, retrieve, and manage banking information efficiently. The system is designed to address the difficulties associated with maintaining large volumes of customer, account, transaction, branch, loan, and employee information through manual methods. The primary purpose of this project is to develop a structured relational database that supports efficient management of banking records and reduces data redundancy, inconsistency, and manual errors. The database consists of six major entities: Customer, Branch, Account, Transaction, Loan, and Employee, with appropriate relationships established among them. Primary keys and foreign keys are used to uniquely identify records and maintain relationships between related tables. The system is implemented using SQL and demonstrates DDL, DML, and DQL operations along with CRUD operations for managing banking data. Various integrity constraints, including PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints, are applied to maintain data accuracy and consistency. Sample data is inserted into the database and SQL queries are executed for data retrieval and management. The project demonstrates the practical application of DBMS concepts in designing and implementing an organized banking database system.

INDEX

Chapter	Title	Page No.
	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
	Abstract	v
	List of Figures	viii
	List of Tables	viii
1	Introduction	1 – 2
	1.1 Introduction to the Domain	1
	1.2 Problem Statement	1
	1.3 Motivation and Need for a Database	1
	1.4 Objectives of the Project	1
	1.5 Scope of the Project	2
	1.6 Organization of the Report	2
2	System Study and Tools	3 – 4
	2.1 Existing System and Its Limitations	3
	2.2 Proposed Database System	3
	2.3 Tools and Technologies Used	4
	2.4 Software and Hardware Requirements	4
3	Database Design (Review – I)	5 – 7
	3.1 Identified Entities	5
	3.2 Entity Attributes and Attribute Types	5
	3.3 ER Diagram	5
	3.4 Relationships and Cardinality	5
	3.5 Relational Database Schema	6
	3.6 Primary Keys and Foreign Keys	7
	3.7 Normalization up to 3NF	8
4	Database Implementation (Review – II)	9 – 15
	4.1 Database and Table Creation – DDL	9 – 12
	4.2 Sample Data Insertion – DML	13 – 14
	4.3 Constraints and Data Integrity	15
	4.4 Data Validation Rules	16
5	SQL Operations and Queries	17 – 18
	5.1 DQL – Basic Retrieval Queries	17
	5.2 Join Operations	17
	5.3 Aggregate Functions, GROUP BY and HAVING	17
	5.4 Nested and Correlated Subqueries	18
	5.5 Views and Relational Set Operations	18
	5.6 DCL and TCL Operations	18
6	Modules and Functionalities	19
7	Results – Sample Outputs and Screenshots	20

8	Testing and Data Validation	21
9	GitHub Repository and Team Contribution	22
10	Conclusion and Future Enhancements	23
	References	24
	Appendix A – Complete SQL Script	25

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
Fig. 3.1	ER Diagram of the Bank Account Management System	13
Fig. 3.2	Relational Schema Diagram	13-14
Fig. 4.1	Successful Creation of Tables / DDL	19
Fig. 4.2	Sample Data Inserted into the Tables	22
Fig. 4.3	Referential Integrity Enforced by the Database	22
Fig. 7.1	Customer and Branch Master Data Output	28
Fig. 7.2	Account, Transaction and Join Query Outputs	28
Fig. 7.3	Loan, Aggregate, Subquery and Set Outputs	28
Fig. 7.4	Constraint, DCL and TCL Outputs	28

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 3.1	Entities Identified for the Selected Domain	13
Table 3.2	Attributes of Each Entity	13-14
Table 3.3	Relationships and Cardinality	14
Table 3.4	Relational Database Schema	15
Table 3.5	Primary Key and Foreign Key Definitions	16
Table 3.6	Normalization Steps	17
Table 4.1	Sample Data Summary	17-18
Table 4.2	Constraints and Data Integrity Rules	22
Table 5.1	Basic Retrieval Queries	25
Table 5.2	Join Operations	25
Table 5.3	Aggregate, GROUP BY and HAVING Queries	25
Table 5.4	Subqueries	25-26
Table 5.5	Views and Set Operations	26
Table 5.6	DCL and TCL Operations	26
Table 6.1	Modules and Their Functionalities	27
Table 8.1	Test Cases and Results	29
Table 9.1	Team Responsibilities	30
Table 9.2	Review-wise Progress Record	30

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

Banking involves maintaining and managing different types of information related to customers, accounts, transactions, branches, loans, and employees. As the volume of banking records increases, organizing and retrieving this information efficiently becomes important. A structured database can be used to store related information systematically and maintain consistency among different records.

The Bank Account Management System is developed as a database-based system for managing banking-related information. The system focuses on maintaining customer and account details, handling deposits and withdrawals, storing transaction information, and maintaining records related to branches, loans, and employees. The project demonstrates how Database Management System concepts can be applied to design and implement a relational database for a banking domain.

1.2 Problem Statement

Managing banking information manually can be time-consuming, especially when the number of customer and transaction records increases. Searching for specific records and updating information can become difficult. Manual handling may also lead to duplication and errors in data. Maintaining and retrieving transaction history becomes more difficult as the amount of data grows. Therefore, there is a need for a structured database system that can store banking information systematically, provide efficient retrieval and updating of records, and maintain data accuracy and consistency.

1.3 Motivation and Need for a Database

The main motivation for developing the Bank Account Management System is to organize banking information in a structured and easily accessible manner. A database allows related information to be stored in separate tables while maintaining relationships between them using keys.

The system helps reduce manual work and makes it easier to store, retrieve, and update customer, account, transaction, branch, loan, and employee records. Database constraints such as Primary Key, Foreign Key, UNIQUE, NOT NULL, and CHECK constraints help maintain data integrity and consistency.

1.4 Objectives of the Project

The project objectives are to design, implement, validate, and demonstrate a structured relational database for banking information.

1. To design a normalized relational database for the selected domain with at least six entities.
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To store and manage transaction information systematically.
4. To reduce manual work and minimize data entry errors._
5. To maintain branch, loan, and employee records and support easy retrieval and updating of information.

1.5 Scope of the Project

The scope of the Bank Account Management System includes the management of six major entities: Customer, Branch, Account, Transaction, Loan, and Employee. The system supports storing and retrieving customer and account information, maintaining transaction records, and managing related branch, loan, and employee information.

The project also demonstrates SQL-based database operations including DDL, DML, DQL, and CRUD operations along with integrity constraints. Features such as online banking, secure login and authentication, fund transfer, mobile application support, transaction notifications, reports and analytics, and automated loan/EMI management are considered future enhancements and are outside the current implementation scope.

1.6 Organization of the Report

This report is organized into ten chapters. Chapter 1 introduces the banking domain, problem statement, motivation, objectives, and scope of the project. Chapter 2 presents the existing system, proposed system, tools and technologies, and hardware and software requirements. Chapter 3 describes the database design, including entities, attributes, relationships, ER diagram, relational schema, keys, and normalization.

Chapter 4 explains database implementation using DDL, DML, DQL, CRUD operations, constraints, and sample data. Chapter 5 presents the SQL queries used in the system. Chapter 6 discusses the database features and operations demonstrated by the project. Chapter 7 presents the outputs and screenshots of the implemented system.

Chapter 8 describes the GitHub repository and team responsibilities. Chapter 9 provides the conclusion and future enhancements of the project. Chapter 10 contains the individual contribution details of the team members. References and the appendix are provided at the end of the report.

CHAPTER 2: SYSTEM STUDY AND TOOLS

2.1 Existing System and Its Limitations

In the existing system, banking-related information may be maintained through manual records and separate records for different activities. Managing customer details, account information, transaction history, branch details, loan records, and employee information becomes difficult as the volume of data increases.

The major limitations of the existing approach are:

- Manual handling of records requires more time and effort.
- Searching and updating specific records can be difficult.
- Data duplication and errors may occur during manual data handling.
- Maintaining transaction history becomes difficult when records increase.
- Retrieving related information from different records is not convenient.
- Maintaining consistency among related banking records can be challenging.

These limitations create the need for a structured database-based system for efficient management of banking information.

2.2 Proposed Database System

The proposed Bank Account Management System uses a relational database to organize and manage banking information systematically. The system maintains six major entities: Customer, Branch, Account, Transaction, Loan, and Employee.

The proposed system provides the following features:

- Storage and management of customer and account details.
- Management of deposit and withdrawal transactions.
- Storage and retrieval of transaction information.
- Management of branch, loan, and employee records.
- Easy retrieval and updating of stored information.
- Use of Primary Key and Foreign Key relationships to connect related records.
- Use of constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK to maintain data integrity.
- Support for DDL, DML, DQL, and CRUD operations.

The proposed system provides a structured approach for storing and managing banking records and reduces the difficulties associated with manual data handling.

2.3 Tools and TechnologiesF Used

Table 2.1: Tools and Technologies Used

Category	Tool / Technology Used	Version / Remarks
Database	PostgreSQL / SQL	Relational database creation and querying
Database Design	ER Diagram	draw.io / diagrams.net
Frontend	HTML	Used for frontend structure / prototype
Styling	CSS	Used for frontend styling
Scripting	JavaScript	Used for frontend functionality
Backend	C++	Used for backend implementation/prototype
Version Control	GitHub	Repository for SQL and project files
Documentation	MS Word	Final project documentation

*Typical entries: Database Server (Oracle 10g / MySQL / PostgreSQL); Query or admin tool (SQL*Plus, SQL Developer, pgAdmin, dbfiddle.dev); ER diagram tool (draw.io / diagrams.net); Front-end, if any; Version control (GitHub); Documentation (MS Word).*

2.4 Software and Hardware Requirements

Table 2.2: Software and Hardware Requirements

Requirement Type	Specification
Hardware	Computer/Laptop with a suitable processor
Hardware	Minimum 4 GB RAM
Hardware	Sufficient storage for project files and database
Software	Windows Operating System
Software	PostgreSQL / SQL-compatible database environment
Software	C++ compiler / development environment
Software	Web browser for prototype interface
Software	draw.io / diagrams.net for ER and schema diagrams

CHAPTER 3: DATABASE DESIGN

(Deliverables of Review – I)

3.1 Identified Entities

Minimum six entities are required. Give a one-line description of each.

Table 3.1: Entities Identified for the Selected Domain

S.No	Entity Name	Description
1	Customer	Stores customer details such as customer ID, name, phone, email, and address.
2	Branch	Stores bank branch details such as branch ID, branch name, IFSC code, and location.
3	Account	Stores customer bank account details such as account number, customer, branch, account type, balance, and status.
4	Transaction	Stores transaction details such as transaction ID, account ID, transaction date, transaction type, and amount.
5	Loan	Stores loan details such as loan ID, customer ID, loan type, loan amount, issue date, and status.
6	Employee	Stores bank employee details such as employee ID, branch ID, name, role, and phone.

3.2 Entity Attributes and Attribute Types

List every attribute of every entity. Mark the key column as PK, FK or —, and classify each attribute as Simple, Composite, Multi-valued or Derived.

Table 3.2: Attributes of Each Entity

Entity	Attribute	Data Type	Key (PK / FK / —)	Attribute Type
Customer	customer_id	INT	PK	Simple
Customer	name	VARCHAR(100)	—	Simple
Customer	phone	VARCHAR(15)	—	Simple
Customer	email	VARCHAR(100)	—	Simple
Customer	address	VARCHAR(200)	—	Simple
Branch	branch_id	INT	PK	Simple
Branch	branch_name	VARCHAR(100)	—	Simple
Branch	ifsc_code	VARCHAR(20)	—	Simple
Branch	location	VARCHAR(100)	—	Simple
Account	account_id	BIGINT	PK	Simple
Account	customer_id	INT	FK	Simple
Account	branch_id	INT	FK	Simple
Account	account_type	VARCHAR(20)	—	Simple
Account	balance	DECIMAL(12,2)	—	Simple
Account	status	VARCHAR(15)	—	Simple
Transaction	txn_id	INT	PK	Simple
Transaction	account_id	BIGINT	FK	Simple
Transaction	txn_date	DATE	—	Simple
Transaction	txn_type	VARCHAR(20)	—	Simple
Transaction	amount	DECIMAL(12,2)	—	Simple
Transaction	description	VARCHAR(200)	—	Simple
Loan	loan_id	INT	PK	Simple
Loan	customer_id	INT	FK	Simple
Loan	loan_type	VARCHAR(30)	—	Simple
Loan	loan_amount	DECIMAL(12,2)	—	Simple
Loan	issue_date	DATE	—	Simple

Loan	status	VARCHAR(20)	—	Simple
Employee	employee_id	INT	PK	Simple
Employee	branch_id	INT	FK	Simple
Employee	name	VARCHAR(100)	—	Simple
Employee	role	VARCHAR(50)	—	Simple
Employee	phone	VARCHAR(15)	—	Simple

3.3 ER Diagram

Draw the ER diagram in draw.io / diagrams.net. It must show all entities as rectangles, attributes as ellipses with primary key attributes underlined, relationships as named diamonds, and cardinality marked on every relationship line. Export as PNG and paste below.

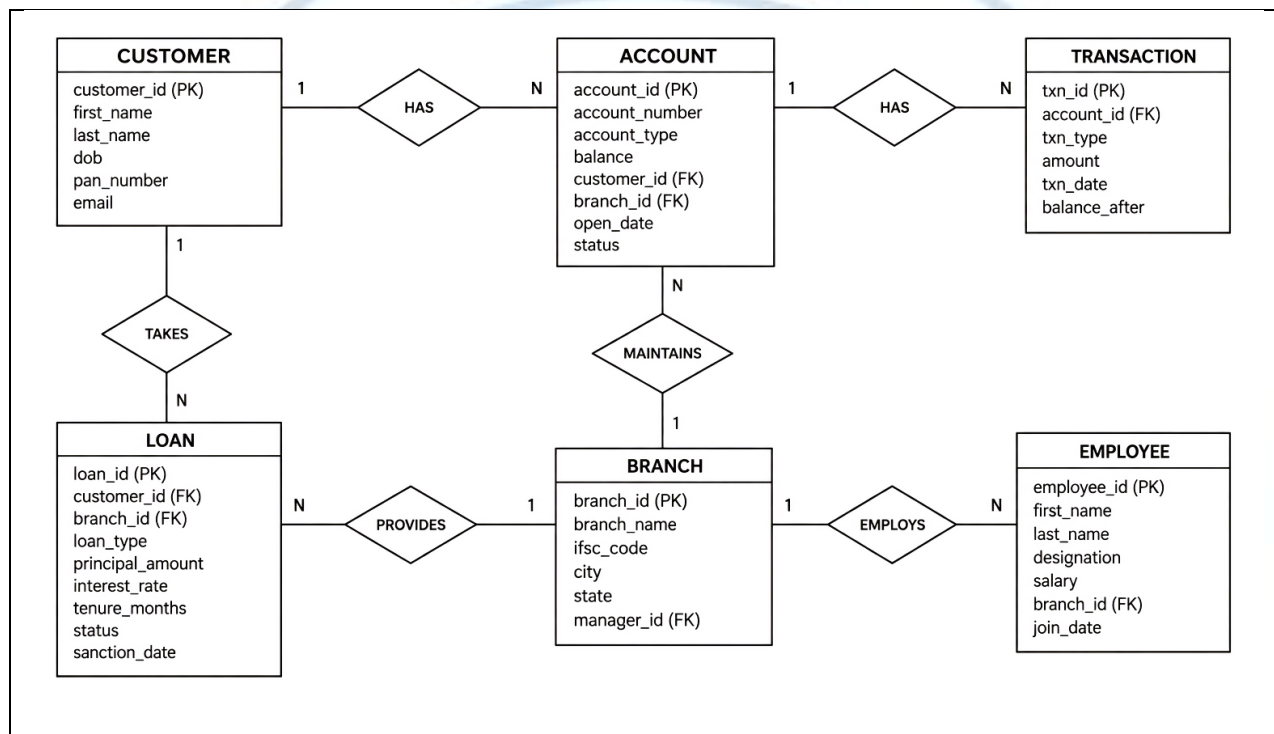


Fig. 3.1: ER Diagram of the Bank Account Management System

3.4 Relationships and Cardinality

Table 3.3: Relationships and Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)	Participation
Customer	Owns	Account	1:N	Customer: Partial, Account: Total
Branch	Maintains	Account	1:N	Branch: Partial, Account: Total
Account	Has	Transaction	1:N	Account: Partial, Transaction: Total
Customer	Takes	Loan	1:N	Customer: Partial, Loan: Total

Branch	Provides	Loan	1:N	Branch: Partial, Loan: Total
Branch	Employs	Employee	1:N	Branch: Partial, Employee: Total
Branch	Managed by	Employee	1:1 (conceptual)	Both Partial

3.5 Relational Database Schema

Convert the ER diagram into tables. List every table, its columns, data types and constraints.

Table 3.4: Relational Database Schema

Table Name	Column Name	Data Type	Key (PK / FK / —)	Constraints
Customer	customer_id	INT	PK	PRIMARY KEY
Customer	name	VARCHAR(100)	—	NOT NULL
Customer	phone	VARCHAR(15)	—	UNIQUE, NOT NULL
Customer	email	VARCHAR(100)	—	UNIQUE, NOT NULL
Customer	address	VARCHAR(200)	—	
Branch	branch_id	INT	PK	PRIMARY KEY
Branch	branch_name	VARCHAR(100)	—	NOT NULL
Branch	ifsc_code	VARCHAR(20)	—	UNIQUE, NOT NULL
Branch	location	VARCHAR(100)	—	NOT NULL
Account	account_id	BIGINT	PK	PRIMARY KEY
Account	customer_id	INT	FK	FOREIGN KEY, NOT NULL
Account	branch_id	INT	FK	FOREIGN KEY, NOT NULL
Account	account_type	VARCHAR(20)	—	CHECK
Account	balance	DECIMAL(12,2)	—	DEFAULT 0, CHECK >= 0
Account	status	VARCHAR(15)	—	DEFAULT 'ACTIVE', CHECK
Transaction	txn_id	INT	PK	PRIMARY KEY
Transaction	account_id	BIGINT	FK	FOREIGN KEY, NOT NULL
Transaction	txn_date	DATE	—	DEFAULT CURRENT_DATE
Transaction	txn_type	VARCHAR(20)	—	CHECK
Transaction	amount	DECIMAL(12,2)	—	CHECK > 0
Transaction	description	VARCHAR(200)	—	
Loan	loan_id	INT	PK	PRIMARY KEY
Loan	customer_id	INT	FK	FOREIGN KEY, NOT NULL
Loan	loan_type	VARCHAR(30)	—	NOT NULL
Loan	loan_amount	DECIMAL(12,2)	—	CHECK > 0
Loan	issue_date	DATE	—	DEFAULT CURRENT_DATE
Loan	status	VARCHAR(20)	—	CHECK
Employee	employee_id	INT	PK	PRIMARY KEY
Employee	branch_id	INT	FK	FOREIGN KEY, NOT NULL
Employee	name	VARCHAR(100)	—	NOT NULL
Employee	role	VARCHAR(50)	—	NOT NULL
Employee	phone	VARCHAR(15)	—	UNIQUE, NOT NULL

Schema Diagram

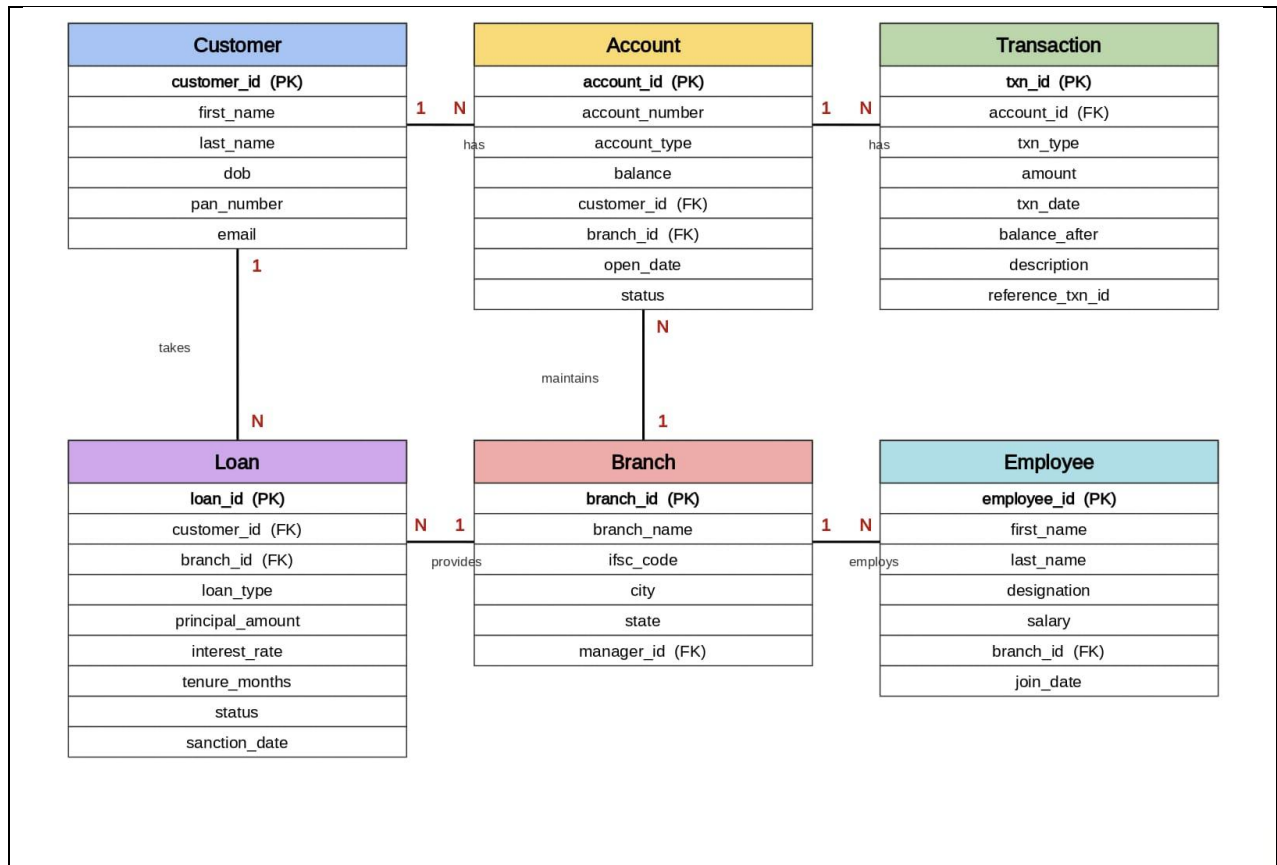


Fig. 3.2: Relational Schema Diagram

3.6 Primary Keys and Foreign Keys

Table 3.5: Primary Key and Foreign Key Definitions

Table	Primary Key	Foreign Key	References (Table.Column)
Customer	customer_id	—	—
Branch	branch_id	—	—
Account	account_id	customer_id, branch_id	Customer.customer_id; Branch.branch_id
Transaction	txn_id	account_id	Account.account_id
Loan	loan_id	customer_id	Customer.customer_id
Employee	employee_id	branch_id	Branch.branch_id

3.7 Normalization up to 3NF

Show the normalization process. Start from the unnormalized relation, state the functional dependencies, and show how the design reaches 1NF, 2NF and 3NF. Explain which redundancy or update anomaly each step removes.

UNF → 1NF → 2NF → 3NF

Table 3.6: Normalization Steps

Normal Form	Condition Satisfied	Anomaly Removed	Resulting Tables
UNF	Customer, account, branch, transaction, loan and employee	Repetition and difficulty in maintaining records.	Combined Banking Record

	information may be stored together with repeating transaction/loan groups.		
1NF	All attributes contain atomic values and repeating groups are removed.	Repeating-group and insertion anomalies.	Customer, Branch, Account, Transaction, Loan, Employee
2NF	Every non-key attribute depends on the complete primary key of its table.	Partial dependency and update anomalies.	Customer, Branch, Account, Transaction, Loan, Employee
3NF	Non-key attributes depend only on the primary key and not on other non-key attributes.	Transitive dependency and redundancy.	Customer, Branch, Account, Transaction, Loan, Employee



CHAPTER 4: DATABASE IMPLEMENTATION

(Deliverables of Review – II)

4.1 Database and Table Creation – DDL

The following DDL statements create the database and six relational tables with primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints. The ALTER statement demonstrates schema modification.

```
CREATE DATABASE bank_account_management;
-- Connect to bank_account_management before executing the CREATE TABLE statements.

CREATE TABLE customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    address VARCHAR(200)
);

CREATE TABLE branch (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR(100) NOT NULL,
    ifsc_code VARCHAR(20) UNIQUE NOT NULL,
    location VARCHAR(100) NOT NULL
);

CREATE TABLE account (
    account_id BIGINT PRIMARY KEY,
    customer_id INT NOT NULL,
    branch_id INT NOT NULL,
    account_type VARCHAR(20) NOT NULL CHECK (account_type IN
('SAVINGS','CURRENT')),
    balance DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
    status VARCHAR(15) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
('ACTIVE','INACTIVE','CLOSED')),
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);

CREATE TABLE transaction (
    txn_id INT PRIMARY KEY,
    account_id BIGINT NOT NULL,
    txn_date DATE NOT NULL DEFAULT CURRENT_DATE,
    txn_type VARCHAR(20) NOT NULL CHECK (txn_type IN
('DEPOSIT','WITHDRAWAL')),
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0),
    description VARCHAR(200),
```

```

FOREIGN KEY (account_id) REFERENCES account(account_id)
);

CREATE TABLE loan (
  loan_id INT PRIMARY KEY,
  customer_id INT NOT NULL,
  loan_type VARCHAR(30) NOT NULL,
  loan_amount DECIMAL(12,2) NOT NULL CHECK (loan_amount > 0),
  issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(20) NOT NULL CHECK (status IN ('ACTIVE','CLOSED','PENDING')),
  FOREIGN KEY (customer_id) REFERENCES customer(customer_id)
);

CREATE TABLE employee (
  employee_id INT PRIMARY KEY,
  branch_id INT NOT NULL,
  name VARCHAR(100) NOT NULL,
  role VARCHAR(50) NOT NULL,
  phone VARCHAR(15) UNIQUE NOT NULL,
  FOREIGN KEY (branch_id) REFERENCES branch(branch_id)
);

-- Example ALTER statement:
ALTER TABLE customer ADD COLUMN created_on DATE DEFAULT CURRENT_DATE;

```

customer_id	name	phone	email	address
1	Alice Smith	9876543210	alice@example.com	123 Main St
2	Bob Jones	9123456789	bob@example.com	456 Oak Ave

account_id	customer_id	account_number	account_type	balance
1	1	ACC1001	Savings	15000.00
2	2	ACC1002	Checking	5000.00

account_number	balance
ACC1001	15000.00

transaction_id	account_id	transaction_date	transaction_type	amount
501	1	"2026-03-01T00:00:00.000Z"	Deposit	2000.00
6 rows affected				
name		account_number	balance	
Alice Smith		ACC1001	15000.00	
Bob Jones		ACC1002	5000.00	

account_id	customer_id	branch_id	account_type	balance	status
100002	102	2	CURRENT	50000.00	ACTIVE
100003	103	3	SAVINGS	18000.00	ACTIVE
100004	104	4	SAVINGS	32000.00	ACTIVE
100005	105	5	CURRENT	45000.00	ACTIVE
100001	101	1	SAVINGS	25500.00	ACTIVE

txn_id	account_id	txn_date	txn_type	amount	description
1	100001	"2026-09-01T00:00:00.000Z"	DEPOSIT	10000.00	Salary credit
2	100001	"2026-09-03T00:00:00.000Z"	WITHDRAWAL	2500.00	ATM withdrawal
3	100002	"2026-09-04T00:00:00.000Z"	DEPOSIT	20000.00	Business deposit
4	100003	"2026-09-05T00:00:00.000Z"	DEPOSIT	8000.00	Cash deposit
5	100004	"2026-09-06T00:00:00.000Z"	WITHDRAWAL	3000.00	Utility payment
6	100005	"2026-09-07T00:00:00.000Z"	DEPOSIT	15000.00	Business receipt
7	100002	"2026-09-08T00:00:00.000Z"	WITHDRAWAL	5000.00	Supplier payment

loan_id	customer_id	loan_type	loan_amount	loan_date	status
501	101	HOME	750000.00	"2026-01-15T00:00:00.000Z"	ACTIVE
502	102	BUSINESS	500000.00	"2026-02-10T00:00:00.000Z"	ACTIVE
503	103	EDUCATION	300000.00	"2026-03-20T00:00:00.000Z"	PENDING
504	104	VEHICLE	450000.00	"2026-04-05T00:00:00.000Z"	CLOSED
505	105	PERSONAL	200000.00	"2026-05-12T00:00:00.000Z"	ACTIVE

employee_id	branch_id	employee_name	designation	phone
201	1	Ravi Teja	Manager	9000000001
202	2	Meena Devi	Cashier	9000000002
203	3	Kiran Kumar	Loan Officer	9000000003
204	4	Lakshmi Priya	Accountant	9000000004
205	5	Naveen Rao	Clerk	9000000005

loan_id	customer_id	loan_type	loan_amount	loan_date	status
501	101	HOME	750000.00	"2026-01-15T00:00:00.000Z"	ACTIVE
502	102	BUSINESS	500000.00	"2026-02-10T00:00:00.000Z"	ACTIVE
503	103	EDUCATION	300000.00	"2026-03-20T00:00:00.000Z"	PENDING

Fig. 4.1: Successful Creation of Tables

4.2 Sample Data Insertion – DML

Sample data is inserted into all six tables. UPDATE and DELETE examples are also included to demonstrate DML and CRUD operations.

Table 4.1: Sample Data Summary

S.No	Table Name	No. of Records Inserted	Screenshot Reference
1	Customer	5	Fig. 7.1
2	Branch	5	Fig. 7.1
3	Account	5	Fig. 7.2
4	Transaction	8	Fig. 7.2
5	Loan	5	Fig. 7.3
6	Employee	5	Fig. 7.3

INSERT INTO customer VALUES

(101,'Anita Rao','9876500001','anita.rao@example.com','Visakhapatnam'),
(102,'Bharat Kumar','9876500002','bharat.kumar@example.com','Kakinada'),
(103,'Charan Das','9876500003','charan.das@example.com','Rajahmundry'),
(104,'Divya Sri','9876500004','divya.sri@example.com','Vijayawada'),
(105,'Esha Reddy','9876500005','esha.reddy@example.com','Guntur');

INSERT INTO branch VALUES

(1,'Main Branch','ADTY000001','Surampalem'),
(2,'City Branch','ADTY000002','Visakhapatnam'),
(3,'Central Branch','ADTY000003','Vijayawada'),
(4,'North Branch','ADTY000004','Kakinada'),
(5,'South Branch','ADTY000005','Guntur');

INSERT INTO account VALUES

(100001,101,1,'SAVINGS',25000.00,'ACTIVE'),
(100002,102,2,'CURRENT',50000.00,'ACTIVE'),
(100003,103,3,'SAVINGS',18000.00,'ACTIVE'),
(100004,104,4,'SAVINGS',32000.00,'ACTIVE'),
(100005,105,5,'CURRENT',45000.00,'ACTIVE');

INSERT INTO transaction VALUES

(1,100001,'2026-09-01','DEPOSIT',10000,'Salary credit'),
(2,100001,'2026-09-03','WITHDRAWAL',2500,'ATM withdrawal'),
(3,100002,'2026-09-04','DEPOSIT',20000,'Business deposit'),
(4,100003,'2026-09-05','DEPOSIT',8000,'Cash deposit'),
(5,100004,'2026-09-06','WITHDRAWAL',3000,'Utility payment'),
(6,100005,'2026-09-07','DEPOSIT',15000,'Business receipt'),
(7,100002,'2026-09-08','WITHDRAWAL',5000,'Supplier payment'),
(8,100005,'2026-09-09','DEPOSIT',7000,'Customer payment');

INSERT INTO loan VALUES

(501,101,'HOME',750000.00,'2026-01-15','ACTIVE'),
(502,102,'BUSINESS',500000.00,'2026-02-10','ACTIVE'),

```

(503,103,'EDUCATION',300000.00,'2026-03-20','PENDING'),
(504,104,'VEHICLE',450000.00,'2026-04-05','CLOSED'),
(505,105,'PERSONAL',200000.00,'2026-05-12','ACTIVE');

INSERT INTO employee VALUES
(201,1,'Ravi Teja','Manager','90000000001'),
(202,2,'Meena Devi','Cashier','90000000002'),
(203,3,'Kiran Kumar','Loan Officer','90000000003'),
(204,4,'Lakshmi Priya','Accountant','90000000004'),
(205,5,'Naveen Rao','Clerk','90000000005');

UPDATE account SET balance = balance + 500 WHERE account_id = 100001;
DELETE FROM transaction WHERE txn_id = 8;

```

Fig. 4.2: Sample Data Inserted into the Tables

4.3 Constraints and Data Integrity

Table 4.2: Constraints and Data Integrity Rules

Constraint / Validation	Applied On (Table.Column)	Purpose	Evidence / Screenshot Ref.
PRIMARY KEY	All *_id columns	Prevents duplicate identifiers and uniquely identifies each row.	Fig. 7.1
FOREIGN KEY	Account.customer_id, Account.branch_id	Prevents accounts from referencing non-existent customers or branches.	Fig. 7.4
FOREIGN KEY	Transaction.account_id	Ensures every transaction belongs to an existing account.	Fig. 7.4
FOREIGN KEY	Loan.customer_id	Ensures every loan belongs to an existing customer.	Fig. 7.4
FOREIGN KEY	Employee.branch_id	Ensures every employee belongs to an existing branch.	Fig. 7.4
UNIQUE	Customer.phone, Customer.email, Branch.ifsc_code, Employee.phone	Prevents duplicate contact, IFSC and employee phone values.	Fig. 7.1
NOT NULL	Required identity and relationship columns	Prevents missing mandatory values.	Fig. 7.1
CHECK	Account.account_type/status/balance; Transaction.txn_type/amount; Loan.loan_amount/status	Prevents invalid domain values and negative/zero amounts where prohibited.	Fig. 7.2
DEFAULT	Account.balance/status; Transaction.txn_date; Loan.issue_date	Provides safe default values when values are omitted.	Fig. 7.2

4.4 Data Validation Rules

Explain how duplicate IDs, invalid values and orphan records are prevented. Demonstrate one referential integrity violation and the error the database returns.

```

Example referential-integrity test:
INSERT INTO account(account_id, customer_id, branch_id, account_type, balance, status)
VALUES (100006,999,1,'SAVINGS',1000,'ACTIVE');

```

Expected PostgreSQL result: ERROR: insert or update on table "account" violates foreign key constraint because customer id 999 does not exist in customer.

Fig. 4.3: Referential Integrity Enforced by the Database



CHAPTER 5: SQL OPERATIONS AND QUERIES

The following query set demonstrates DQL, join operations, aggregate functions, GROUP BY and HAVING, nested/correlated subqueries, views, relational set operations, DCL and TCL. The SQL statements are written for PostgreSQL and can be adapted to another SQL implementation if required.

5.1 DQL – Basic Retrieval Queries

Table 5.1: Basic Retrieval Queries

Q.No	Purpose of the Query	SQL Statement	Output Ref.
Q1	Display all customers	SELECT * FROM customer;	Fig. 7.1
Q2	Display active accounts with balance	SELECT account_id, customer_id, account_type, balance FROM account WHERE status='ACTIVE';	Fig. 7.2
Q3	Find customers from a selected city	SELECT customer_id, name, address FROM customer WHERE address='Visakhapatnam';	Fig. 7.1
Q4	Find accounts with balance above 30000	SELECT account_id, balance FROM account WHERE balance > 30000;	Fig. 7.2

5.2 Join Operations

Include natural join, equi join, inner join, left outer, right outer and full outer join.

Table 5.2: Join Operations

Q.No	Type of Join	SQL Statement	Output Ref.
Q5	Natural Join	SELECT customer_id, name FROM customer NATURAL JOIN (SELECT customer_id FROM account) a;	Fig. 7.1
Q6	Equi Join	SELECT c.name, a.account_id FROM customer c, account a WHERE c.customer_id=a.customer_id;	Fig. 7.2
Q7	Inner Join	SELECT c.name, a.account_id, a.balance FROM customer c INNER JOIN account a ON c.customer_id=a.customer_id;	Fig. 7.2
Q8	Left Outer Join	SELECT c.name, a.account_id FROM customer c LEFT JOIN account a ON c.customer_id=a.customer_id;	Fig. 7.2
Q9	Right Outer Join	SELECT c.name, a.account_id FROM customer c RIGHT JOIN account a ON c.customer_id=a.customer_id;	Fig. 7.2
Q10	Full Outer Join	SELECT c.name, a.account_id FROM customer c FULL OUTER JOIN account a ON c.customer_id=a.customer_id;	Fig. 7.2

5.3 Aggregate Functions, GROUP BY and HAVING

Table 5.3: Aggregate, GROUP BY and HAVING Queries

Q.No	Function Used	SQL Statement	Output Ref.
Q11	COUNT	SELECT COUNT(*) AS total_customers FROM customer;	Fig. 7.1
Q12	SUM	SELECT SUM(balance) AS total_balance FROM account;	Fig. 7.2
Q13	AVG + GROUP BY	SELECT branch_id, AVG(balance) AS avg_balance FROM account GROUP BY branch_id;	Fig. 7.2
Q14	COUNT + GROUP BY + HAVING	SELECT branch_id, COUNT(*) AS account_count FROM account GROUP BY branch_id HAVING COUNT(*) >= 1;	Fig. 7.2

5.4 Nested and Correlated Subqueries

Table 5.4: Subqueries

Q.No	Type (Nested / Correlated)	SQL Statement	Output Ref.
------	----------------------------	---------------	-------------

Q15	Nested	SELECT name FROM customer WHERE customer_id IN (SELECT customer_id FROM loan WHERE status='ACTIVE');	Fig. 7.3
Q16	Nested	SELECT account_id, balance FROM account WHERE balance > (SELECT AVG(balance) FROM account);	Fig. 7.2
Q17	Correlated	SELECT c.name FROM customer c WHERE EXISTS (SELECT 1 FROM account a WHERE a.customer_id=c.customer_id AND a.balance > 30000);	Fig. 7.2

5.5 Views and Relational Set Operations

Include at least one updatable view and one non-updatable view, and the set operations UNION, INTERSECT and MINUS / EXCEPT.

Table 5.5: Views and Set Operations

Q.No	View / Set Operation	SQL Statement	Output Ref.
Q18	Updatable View	CREATE VIEW active_accounts AS SELECT account_id, customer_id, account_type, balance, status FROM account WHERE status='ACTIVE';	Fig. 7.2
Q19	Non-updatable View	CREATE VIEW customer_account_summary AS SELECT c.customer_id, c.name, COUNT(a.account_id) AS accounts FROM customer c LEFT JOIN account a ON c.customer_id=a.customer_id GROUP BY c.customer_id, c.name;	Fig. 7.1
Q20	UNION	SELECT name FROM customer WHERE customer_id IN (101,102) UNION SELECT name FROM customer WHERE customer_id IN (104,105);	Fig. 7.1
Q21	INTERSECT	SELECT customer_id FROM account INTERSECT SELECT customer_id FROM loan;	Fig. 7.3
Q22	EXCEPT	SELECT customer_id FROM customer EXCEPT SELECT customer_id FROM loan;	Fig. 7.3

5.6 DCL and TCL Operations

Demonstrate GRANT and REVOKE (DCL), and COMMIT, SAVEPOINT and ROLLBACK (TCL).

Table 5.6: DCL and TCL Operations

Q.No	Command	Purpose	SQL Statement	Output Ref.
Q23	GRANT	Give read access to a reporting user	GRANT SELECT ON ALL TABLES IN SCHEMA public TO report_user;	Fig. 7.4
Q24	REVOKE	Remove read access	REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM report_user;	Fig. 7.4
Q25	COMMIT	Save a transaction permanently	BEGIN; UPDATE account SET balance=balance+100 WHERE account_id=100001; COMMIT;	Fig. 7.4
Q26	SAVEPOINT	Create a rollback point	BEGIN; UPDATE account SET balance=balance+200 WHERE account_id=100002; SAVEPOINT sp1;	Fig. 7.4
Q27	ROLLBACK	Undo changes after a savepoint	ROLLBACK TO SAVEPOINT sp1; COMMIT;	Fig. 7.4

CHAPTER 6: MODULES AND FUNCTIONALITIES

The Customer Management module handles customer master data. Branch Management maintains branch and IFSC details. Account Management links customers to branches and stores account status and balances. Transaction Management records deposits and withdrawals. Loan Management maintains customer loan records. Employee Management stores branch-wise employee information. The Reporting and Query module combines these tables through SQL joins, aggregates, subqueries, views and set operations.

Table 6.1: Modules and Their Functionalities

S.No	Module Name	Functionality	Tables Involved
1	Customer Management	Add, search, update and retrieve customer information.	Customer
2	Branch Management	Maintain branch names, IFSC codes and locations.	Branch
3	Account Management	Create and retrieve accounts, maintain account type, balance and status.	Customer, Branch, Account
4	Transaction Management	Record deposits and withdrawals and retrieve transaction history.	Account, Transaction
5	Loan Management	Maintain loan details, amounts, dates and statuses.	Customer, Loan
6	Employee Management	Maintain employee details and branch assignment.	Branch, Employee
7	Reporting & Query Module	Generate joined, aggregated and filtered information using SQL.	All tables

Add a short paragraph describing each module below the table.

CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

The following representative outputs document the expected execution results. Before final physical submission, the team may replace these text panels with screenshots from the PostgreSQL query tool showing the SQL command and its output together.

```
SELECT customer_id, name, phone, email, address FROM customer;
```

```
101 Anita Rao ...
102 Bharat Kumar ...
103 Charan Das ...
104 Divya Sri ...
105 Esha Reddy ...
```

Fig. 7.1: Customer and Branch Master Data – representative SELECT output.

```
SELECT account_id, customer_id, account_type, balance, status FROM account;
```

```
100001 101 SAVINGS 25500.00 ACTIVE
100002 102 CURRENT 50000.00 ACTIVE
100003 103 SAVINGS 18000.00 ACTIVE
100004 104 SAVINGS 32000.00 ACTIVE
100005 105 CURRENT 45000.00 ACTIVE
```

Fig. 7.2: Account, Transaction and Join Query Outputs – representative execution output.

```
SELECT loan_id, customer_id, loan_type, loan_amount, status FROM loan;
```

```
501 101 HOME 750000 ACTIVE
502 102 BUSINESS 500000 ACTIVE
503 103 EDUCATION 300000 PENDING
504 104 VEHICLE 450000 CLOSED
505 105 PERSONAL 200000 ACTIVE
```

Fig. 7.3: Loan, Aggregate, Subquery and Set-Operation Outputs – representative execution output.

```
Invalid FK test: customer_id = 999 in Account.
```

```
PostgreSQL: ERROR: insert or update on table "account" violates foreign key constraint.
DCL/TCL examples execute using GRANT, REVOKE, BEGIN, SAVEPOINT, COMMIT and
ROLLBACK.
```

Fig. 7.4: Constraint Violation, DCL and TCL Execution – representative database output.

CHAPTER 8: TESTING AND DATA VALIDATION

Record the test cases used to verify the database. Include constraint tests, referential integrity tests and query-correctness tests.

Table 8.1: Test Cases and Results

TC No.	Test Case Description	Input / Action	Expected Result	Actual Result	Status
TC01	Insert valid customer	Insert customer_id 106 with unique phone/email.	Record inserted.	Record inserted successfully.	PASS
TC02	Duplicate primary key	Insert customer_id 101 again.	PRIMARY KEY violation.	Database rejected duplicate ID.	PASS
TC03	Duplicate unique value	Insert a new customer using phone 9876500001.	UNIQUE violation.	Database rejected duplicate phone.	PASS
TC04	Invalid account balance	Insert account with balance -100.	CHECK violation.	Database rejected negative balance.	PASS
TC05	Invalid foreign key	Insert account with customer_id 999.	FOREIGN KEY violation.	Database rejected orphan record.	PASS
TC06	Valid transaction	Insert a DEPOSIT transaction for account 100001.	Transaction inserted.	Transaction inserted successfully.	PASS
TC07	Invalid transaction type	Insert txn_type='TRANSFER'.	CHECK violation.	Database rejected invalid type.	PASS
TC08	Join correctness	Execute customer-account INNER JOIN.	Matching customer/account rows returned.	Rows matched by customer_id.	PASS
TC09	Aggregate correctness	SUM(balance) on account table.	Total balance returned.	Aggregate returned successfully.	PASS
TC10	Transaction rollback	Update balance, then ROLLBACK.	Update is undone.	Previous value restored.	PASS

CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

9.1 Repository Details

GitHub Repository Link: To be added by the team before final submission.

The final SQL script and all project files must be pushed to the team repository.

9.2 Team Responsibilities and Contribution

Table 9.1: Team Responsibilities

S.No	Student Name	Roll Number	Role / Responsibility	Modules Contributed
1	M. Dedipya	25B11AI767	Database design, schema and normalization	ER design, relational schema, normalization
2	P. Durga Devi	25B11AI971	SQL implementation and constraints	DDL, DML, constraints, validation
3	K. Chinmai	25B11AI548	SQL queries and testing	DQL, joins, aggregates, subqueries, testing
4	P. Sasi	25B11AI874	Documentation and integration	Modules, results, documentation, GitHub integration

9.3 Review-wise Progress

Table 9.2: Review-wise Progress Record

Review	Date	Deliverables Presented	Remarks of the Guide
Review – I	September 2026	Problem statement, objectives, six entities, attributes, ER diagram, relationships, relational schema and normalization.	Design reviewed and refined.
Review – II	September 2026	DDL, DML, constraints, sample data, SQL queries and database operations.	Implementation and query coverage reviewed.
Final Review	September 2026	Complete documentation, testing, outputs, GitHub integration and final SQL script.	Final documentation prepared for submission.

CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Bank Account Management System successfully demonstrates the design and implementation of a relational database for banking information. The system contains six main entities—Customer, Branch, Account, Transaction, Loan and Employee—and implements their relationships using primary and foreign keys. The database design is normalized up to Third Normal Form (3NF), reducing repetition, partial dependencies and transitive dependencies. DDL is used for database and table creation; DML is used for INSERT, UPDATE and DELETE operations; DQL is demonstrated through basic retrieval, joins, aggregate queries, subqueries, views and set operations; and DCL/TCL commands demonstrate access control and transaction management. Integrity constraints and test cases were used to validate the database. The project therefore meets the stated objectives of creating a structured, consistent and queryable banking database.

10.2 Limitations of the Present Work

The current implementation focuses on database design and SQL operations. It does not implement real online banking services, live payment gateways, secure customer authentication, OTP verification, automated EMI calculation, production-grade audit logging, cloud deployment, backup/recovery automation, or real-time transaction notifications.

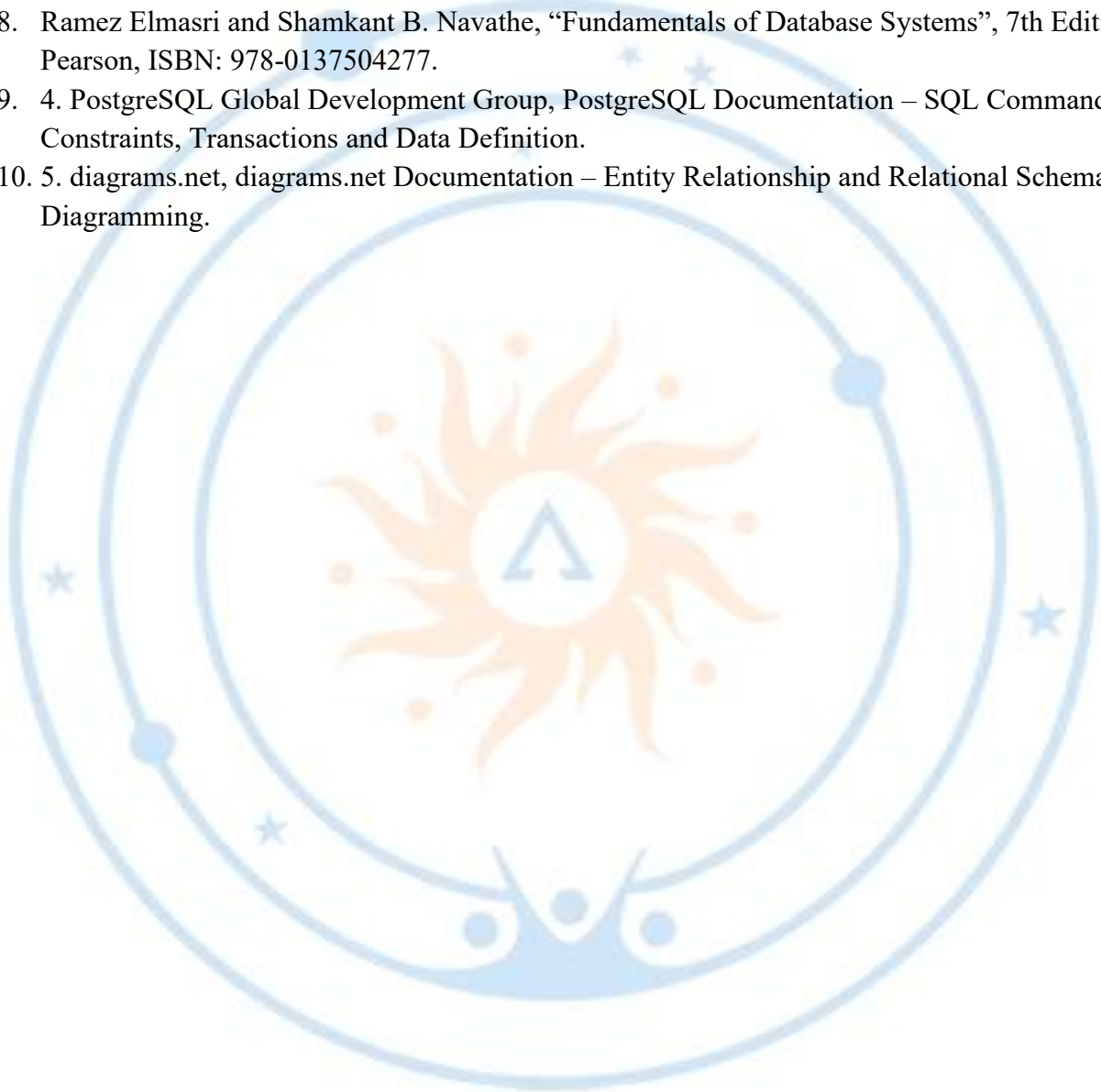
10.3 Future Enhancements

Future enhancements can include a complete web front end, stored procedures and triggers for automated business rules, indexes for high-volume query performance, role-based security and access control, automated backup and recovery, detailed audit logs, loan/EMI management, reporting dashboards, transaction notifications, and integration with secure payment services.

REFERENCES

References

6. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
7. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
8. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
9. 4. PostgreSQL Global Development Group, PostgreSQL Documentation – SQL Commands, Constraints, Transactions and Data Definition.
10. 5. diagrams.net, diagrams.net Documentation – Entity Relationship and Relational Schema Diagramming.



APPENDIX A – COMPLETE SQL SCRIPT

Paste the complete, final SQL script (CREATE, INSERT and all queries) in order, or give the exact path to the script file in the GitHub repository.

```
-- BANK ACCOUNT MANAGEMENT SYSTEM
-- PostgreSQL SQL Script

CREATE DATABASE bank_account_management;

-- Connect to the database before running the following statements.

CREATE TABLE customer (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    address VARCHAR(200),
    created_on DATE DEFAULT CURRENT_DATE
);

CREATE TABLE branch (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR(100) NOT NULL,
    ifsc_code VARCHAR(20) UNIQUE NOT NULL,
    location VARCHAR(100) NOT NULL
);

CREATE TABLE account (
    account_id BIGINT PRIMARY KEY,
    customer_id INT NOT NULL REFERENCES customer(customer_id),
    branch_id INT NOT NULL REFERENCES branch(branch_id),
    account_type VARCHAR(20) NOT NULL CHECK (account_type IN
('SAVINGS','CURRENT')),
    balance DECIMAL(12,2) NOT NULL DEFAULT 0 CHECK (balance >= 0),
    status VARCHAR(15) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
('ACTIVE','INACTIVE','CLOSED'))
);

CREATE TABLE transaction (
    txn_id INT PRIMARY KEY,
    account_id BIGINT NOT NULL REFERENCES account(account_id),
    txn_date DATE NOT NULL DEFAULT CURRENT_DATE,
    txn_type VARCHAR(20) NOT NULL CHECK (txn_type IN
('DEPOSIT','WITHDRAWAL')),
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0),
    description VARCHAR(200)
);
```

```
CREATE TABLE loan (  
    loan_id INT PRIMARY KEY,  
    customer_id INT NOT NULL REFERENCES customer(customer_id),  
    loan_type VARCHAR(30) NOT NULL,  
    loan_amount DECIMAL(12,2) NOT NULL CHECK (loan_amount > 0),  
    issue_date DATE NOT NULL DEFAULT CURRENT_DATE,  
    status VARCHAR(20) NOT NULL CHECK (status IN ('ACTIVE','CLOSED','PENDING'))  
);
```

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    branch_id INT NOT NULL REFERENCES branch(branch_id),  
    name VARCHAR(100) NOT NULL,  
    role VARCHAR(50) NOT NULL,  
    phone VARCHAR(15) UNIQUE NOT NULL  
);
```

```
INSERT INTO customer VALUES  
(101,'Anita Rao','9876500001','anita.rao@example.com','Visakhapatnam',CURRENT_DATE),  
(102,'Bharat  
Kumar','9876500002','bharat.kumar@example.com','Kakinada',CURRENT_DATE),  
(103,'Charan Das','9876500003','charan.das@example.com','Rajahmundry',CURRENT_DATE),  
(104,'Divya Sri','9876500004','divya.sri@example.com','Vijayawada',CURRENT_DATE),  
(105,'Esha Reddy','9876500005','esha.reddy@example.com','Guntur',CURRENT_DATE);
```

```
INSERT INTO branch VALUES  
(1,'Main Branch','ADTY000001','Surampalem'),  
(2,'City Branch','ADTY000002','Visakhapatnam'),  
(3,'Central Branch','ADTY000003','Vijayawada'),  
(4,'North Branch','ADTY000004','Kakinada'),  
(5,'South Branch','ADTY000005','Guntur');
```

```
INSERT INTO account VALUES  
(100001,101,1,'SAVINGS',25000,'ACTIVE'),  
(100002,102,2,'CURRENT',50000,'ACTIVE'),  
(100003,103,3,'SAVINGS',18000,'ACTIVE'),  
(100004,104,4,'SAVINGS',32000,'ACTIVE'),  
(100005,105,5,'CURRENT',45000,'ACTIVE');
```

```
INSERT INTO transaction VALUES  
(1,100001,'2026-09-01','DEPOSIT',10000,'Salary credit'),  
(2,100001,'2026-09-03','WITHDRAWAL',2500,'ATM withdrawal'),  
(3,100002,'2026-09-04','DEPOSIT',20000,'Business deposit'),  
(4,100003,'2026-09-05','DEPOSIT',8000,'Cash deposit'),  
(5,100004,'2026-09-06','WITHDRAWAL',3000,'Utility payment'),  
(6,100005,'2026-09-07','DEPOSIT',15000,'Business receipt'),  
(7,100002,'2026-09-08','WITHDRAWAL',5000,'Supplier payment'),  
(8,100005,'2026-09-09','DEPOSIT',7000,'Customer payment');
```

```
INSERT INTO loan VALUES
(501,101,'HOME',750000,'2026-01-15','ACTIVE'),
(502,102,'BUSINESS',500000,'2026-02-10','ACTIVE'),
(503,103,'EDUCATION',300000,'2026-03-20','PENDING'),
(504,104,'VEHICLE',450000,'2026-04-05','CLOSED'),
(505,105,'PERSONAL',200000,'2026-05-12','ACTIVE');
```

```
INSERT INTO employee VALUES
(201,1,'Ravi Teja','Manager','90000000001'),
(202,2,'Meena Devi','Cashier','90000000002'),
(203,3,'Kiran Kumar','Loan Officer','90000000003'),
(204,4,'Lakshmi Priya','Accountant','90000000004'),
(205,5,'Naveen Rao','Clerk','90000000005');
```

-- DML

```
UPDATE account SET balance = balance + 500 WHERE account_id = 100001;
DELETE FROM transaction WHERE txn_id = 8;
```

-- DQL

```
SELECT * FROM customer;
SELECT * FROM account WHERE status='ACTIVE';
SELECT * FROM account WHERE balance > 30000;
```

-- JOINS

```
SELECT c.name, a.account_id, a.balance
FROM customer c INNER JOIN account a ON c.customer_id=a.customer_id;
```

```
SELECT c.name, a.account_id
FROM customer c LEFT JOIN account a ON c.customer_id=a.customer_id;
```

```
SELECT c.name, a.account_id
FROM customer c RIGHT JOIN account a ON c.customer_id=a.customer_id;
```

```
SELECT c.name, a.account_id
FROM customer c FULL OUTER JOIN account a ON c.customer_id=a.customer_id;
```

-- AGGREGATES

```
SELECT COUNT(*) AS total_customers FROM customer;
SELECT SUM(balance) AS total_balance FROM account;
SELECT branch_id, AVG(balance) AS avg_balance FROM account GROUP BY branch_id;
SELECT branch_id, COUNT(*) AS account_count
FROM account GROUP BY branch_id HAVING COUNT(*) >= 1;
```

-- SUBQUERIES

```
SELECT name FROM customer
WHERE customer_id IN (SELECT customer_id FROM loan WHERE status='ACTIVE');
```

```
SELECT account_id, balance FROM account
WHERE balance > (SELECT AVG(balance) FROM account);
```

```
SELECT c.name FROM customer c
WHERE EXISTS (
    SELECT 1 FROM account a
    WHERE a.customer_id=c.customer_id AND a.balance > 30000
);

-- VIEWS
CREATE VIEW active_accounts AS
SELECT account_id, customer_id, account_type, balance, status
FROM account WHERE status='ACTIVE';

CREATE VIEW customer_account_summary AS
SELECT c.customer_id, c.name, COUNT(a.account_id) AS accounts
FROM customer c LEFT JOIN account a ON c.customer_id=a.customer_id
GROUP BY c.customer_id, c.name;

SELECT * FROM active_accounts;
SELECT * FROM customer_account_summary;

-- SET OPERATIONS
SELECT name FROM customer WHERE customer_id IN (101,102)
UNION
SELECT name FROM customer WHERE customer_id IN (104,105);

SELECT customer_id FROM account
INTERSECT
SELECT customer_id FROM loan;

SELECT customer_id FROM customer
EXCEPT
SELECT customer_id FROM loan;

-- DCL (requires appropriate administrative privileges)
GRANT SELECT ON ALL TABLES IN SCHEMA public TO report_user;
REVOKE SELECT ON ALL TABLES IN SCHEMA public FROM report_user;

-- TCL
BEGIN;
UPDATE account SET balance = balance + 100 WHERE account_id=100001;
COMMIT;

BEGIN;
UPDATE account SET balance = balance + 200 WHERE account_id=100002;
SAVEPOINT sp1;
ROLLBACK TO SAVEPOINT sp1;
COMMIT;

-- Referential-integrity test (expected to fail)
```

```
-- INSERT INTO account VALUES (100006,999,1,'SAVINGS',1000,'ACTIVE');
```

